

Inheritance · Polymorphism Abstract Classes · Interfaces

Extending types · Dynamic behavior · Contracts for design

Faculty of Information Technologies
Industrial University of Ho Chi Minh City

Chapter Contents

01

Inheritance

super · override · protected · final

02

Polymorphism

Upcasting · dispatch · instanceof

03

Abstract Classes

Abstract methods · shared base

04

Interfaces

Contracts · multiple · default methods

05

Comparison & Design

When to use what · put it together

01 · INHERITANCE

Inheritance

What · super · protected · override · final

Why Do We Need Inheritance?

Imagine FullTimeEmployee and PartTimeEmployee. Both have employeeId, name, both need pay(). Without inheritance you copy fields and methods — and fix bugs twice.

Duplication

Same name, same validation, same some methods in every class

Hard to change

One rule change means editing every employee type

No shared type

Cannot write one payroll loop for all kinds of employees

Finding the Commonality

Good inheritance starts by asking: what is the same? What is different?

Common

name · identity · print employee information · belong to the company

Different

How pay is calculated — monthly salary vs $\text{hourlyRate} \times \text{hoursWorked}$

Design move

Pull the common part up; leave the different part to subclasses

The Inheritance Solution

Put shared structure in Employee. Specialize FullTime and PartTime. One place for shared code; many forms for pay().

Employee (base)

Holds employeeId, name, constructor, shared helpers

FullTime / PartTime

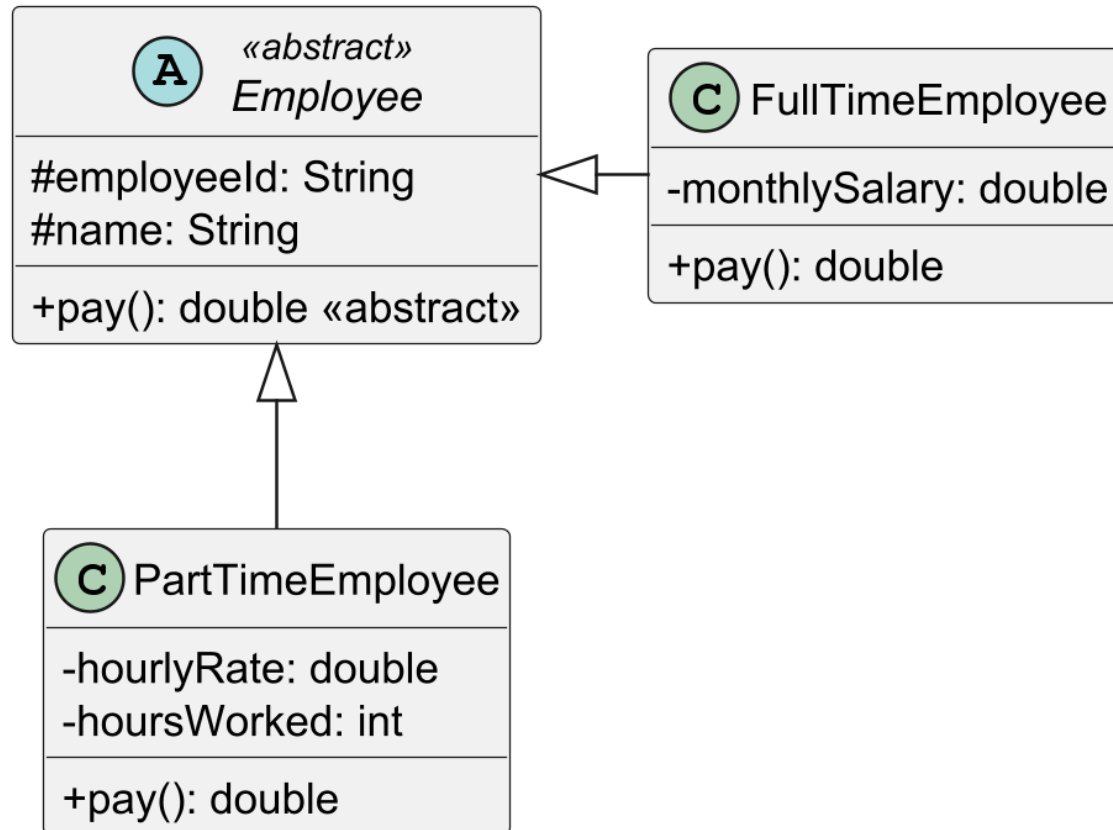
Only implement what is unique — their pay formula

Result

Less duplication · one type for collections · easier evolution

1.1 What Is Inheritance?

Inheritance lets a class reuse and extend another class. Subclass is a specialized version of the superclass — is-a, not has-a.



is-a relationship

FullTimeEmployee
└ is-a → Employee

PartTimeEmployee
└ is-a → Employee

extends Employee

1.1 Inheritance — Code Shape

Case study: Employee · FullTimeEmployee · PartTimeEmployee

Employee

```
public class Employee {  
  
    protected String employeeId;  
    protected String name;  
  
    public abstract double pay();  
}
```

FullTimeEmployee

```
public class FullTimeEmployee  
    extends Employee {  
  
    private double monthlySalary;  
  
    @Override  
    public double pay() {  
        return monthlySalary;  
    }  
}
```

PartTimeEmployee

```
public class PartTimeEmployee  
    extends Employee {  
  
    private double hourlyRate;  
    private int hoursWorked;  
  
    @Override  
    public double pay() {  
        return hourlyRate *  
                               hoursWorked;  
    }  
}
```

The extends Relationship

extends means is-a: FullTimeEmployee is an Employee. PartTimeEmployee is an Employee.
Not has-a.

Syntax

```
class FullTimeEmployee extends Employee { ... }  
class PartTimeEmployee extends Employee { ... }
```

UML

Arrows from both subclasses up to Employee (generalization)

Promise

Wherever an Employee is expected, FullTime or PartTime may appear

Case study

One hierarchy — used from inheritance through interfaces

1.2 Calling the Superclass Constructor

Subclass constructor must initialize the superclass part

Every object has a full chain of constructors — superclass first

`super(...)`

Explicit call to a superclass constructor — must be the first statement

If you omit `super(...)`

Java inserts `super()` (no-arg) automatically — fails if superclass has none

Order of execution

Superclass constructor body runs completely, then subclass body

1.2 super(...) — Demo

Employee (superclass)

```
public class Employee {  
    protected String employeeId; protected String name;  
    public Employee(String employeeId, String name) { this, employeeId = employeeId; this.name = name; }  
}
```

FullTimeEmployee

```
public class FullTimeEmployee extends Employee {  
    private double monthlySalary;  
  
    public FullTimeEmployee(String employeeId,  
        String name, double monthlySalary) {  
        super(employeeId, name); // first statement  
        this.monthlySalary = monthlySalary;  
    }  
}
```

PartTimeEmployee

```
public class PartTimeEmployee extends Employee {  
    private double hourlyRate ;  
    private int hoursWorked;  
  
    public PartTimeEmployee(String employeeId,  
        String name, double rate, int hours) {  
        super(employeeId, name); // first statement  
        this.hourlyRate = rate;  
        this.hoursWorked = hours;  
    }  
}
```

Constructor Chaining

Creating a subclass object always runs superclass constructors first — the chain from Object down to the class you new.

Order

Object → ... → Employee → FullTimeEmployee

Why

The subclass part sits on top of a fully built superclass part

You control the link

super(...) chooses which superclass constructor runs

What If `super()` Is Missing?

If you do not write `super(...)`, Java inserts `super()` — the no-arg superclass constructor.

If Employee has only `Employee(String, String)`

`super()` does not exist → compile error in subclass

Fix

Call `super(employeeId, name)` explicitly as the first statement

Lesson

Always think about how the superclass part is initialized

Object Structure — One Object, Layered Parts

```
new FullTimeEmployee("An", 15_000_000)
```

FullTimeEmployee object

Employee part

```
name = "An"  
employeeId = ...
```

FullTimeEmployee part

```
monthlySalary = 15,000,000
```

Constructor execution

FullTimeEmployee(...)



super(...)



Employee(...)



FullTimeEmployee body

1.3 Inherited Members

public

Inherited and visible everywhere

protected

Inherited; visible in subclass (and same package)

(package)

Inherited only if subclass is in the same package

private

Not inherited — subclass cannot access directly

Constructors

Not inherited — called via `super(...)`

static

Not instance inheritance; accessed via class name

1.3 What the Subclass Sees

```
public class FullTimeEmployee extends Employee {  
    @Override  
    public String toString() {  
        return employeeId + " - " + name; // protected fields  
    }  
  
    @Override  
    public double pay() { return monthlySalary; }  
}
```

```
FullTimeEmployee e = new FullTimeEmployee("E01", "An", 15_000_000);  
System.out.println(e); // calls toString()  
e.pay();                // FullTime implementation  
// private monthlySalary: visible only inside FullTimeEmployee
```

What Does a Subclass Actually Get?

Inheritance is selective. Not every member of the superclass becomes usable in the subclass the same way.

Gets

public and protected instance members (and package-private if same package)

Does not get

private members as accessible names · constructors · (true) static inheritance

Still in the object

private fields exist in memory; only access path is blocked

Inheritance ≠ Access to Everything

"I extend Employee" does not mean "I can touch every field." Encapsulation still applies.

private stays private

Subclass must use public/protected API of the superclass

Why it matters

Base class can change private representation without breaking subclasses

Design tip

Expose protected methods when subclasses need collaboration — not raw fields if avoidable

Why protected?

private is too closed for collaboration with subclasses. public is too open for the world. protected is the middle ground.

Visible to subclasses

Even in another package — intentional family access

Not a public API

Callers outside the hierarchy should not rely on it

Example

protected String name; — FullTime can read it; random UI code should not

Inherited Members — What the Subclass Can Use

Employee «abstract»

protected employeeId, name	✓	subclass OK
abstract pay()	✓	must implement
package-private member	?	same package only
private field (if any)	X	no direct access
Employee(...)	X	via super(...) only



FullTimeEmployee · PartTimeEmployee

1.4 Protected Access

protected members are visible inside the class, same package, and subclasses (even in other packages). Ideal for fields/methods meant to be reused by inheritance.

private

Only this class

(package)

Same package

protected

Same package +
subclasses

public

Everywhere

Rule of thumb

Use private by default. Use protected when subclasses need direct access. Prefer protected methods over protected fields when possible.

1.4 Protected — Demo

```
public abstract class Employee {  
    protected String employeeId; // visible to subclasses  
    protected String name;      // visible to subclasses  
    // no private identity fields exposed here  
    public abstract double pay();  
}  
  
public class FullTimeEmployee extends Employee {  
    private double monthlySalary; // hidden from other classes  
  
    @Override  
    public String toString() { return employeeId + " - " + name; // OK — protected }  
  
    @Override  
    public double pay() { return monthlySalary; }  
}
```

super Means “The Superclass Part”

super is not magic. It names the superclass view of this object — constructor, field, or method.

super(...)

Run a superclass constructor

super.method()

Call the superclass implementation (often from an override)

super.field

Reach a superclass field hidden by a subclass field of the same name

1.5 The super Keyword

`super(...)`

Call a superclass constructor — must be the first statement

`super.field`

Access a superclass field hidden by a subclass field of the same name

`super.method()`

Call the superclass version of a method (often from an override)

Why needed?

When names clash, or you want both old behavior and new behavior

super — The Superclass Part of this

this



FullTimeEmployee
monthlySalary
toString()

super



Employee
employeeId
name
toString()

super(...)

→ superclass constructor

super.field

→ superclass field

super.method()

→ superclass implementation

1.5 super — Demo

```
public abstract class Employee {  
    protected String employeeId;  
    protected String name;  
    public Employee(String id, String name) { this.employeeId = id; this.name = name; }  
    public String toString() { return employeeId + " - " + name; }  
    public abstract double pay();  
}  
  
public class FullTimeEmployee extends Employee {  
    private double monthlySalary;  
    public FullTimeEmployee(String id, String name, double monthlySalary) {  
        super(id, name); // superclass constructor  
        this.monthlySalary = monthlySalary;  
    }  
    @Override  
    public String toString() { return super.toString() + " (full-time)"; } // superclass method  
    @Override public double pay() { return monthlySalary; }  
}
```

Same Method, Different Behavior

Overriding is the heart of polymorphic behavior: one message, many responses.

Same signature

pay() on Employee — FullTime and PartTime each supply a body

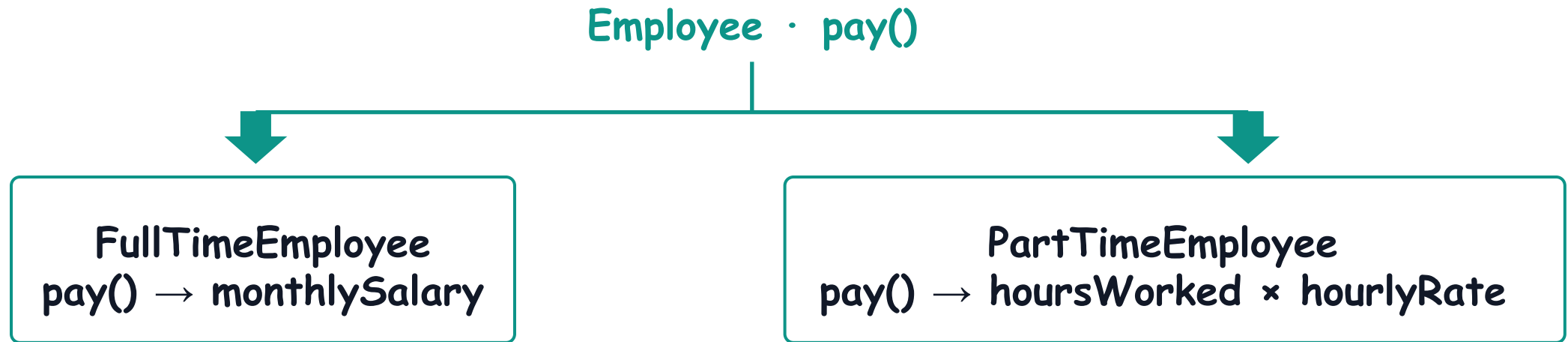
Runtime choice

Which body runs depends on the actual object, not the variable type

Story so far

Inheritance shares structure; overriding specializes behavior

Same Call, Different Behavior → Polymorphism



```
Employee e1 = new FullTimeEmployee(...);  
Employee e2 = new PartTimeEmployee(...);
```

```
e1.pay(); // FullTime formula  
e2.pay(); // PartTime formula
```

Same call

Different
behavior

→ Polymorphism

1.6 Overriding — Demo

```
public class Employee {  
    public double pay() { return 0; }  
}
```

```
public class FullTimeEmployee extends Employee {  
    @Override  
    public double pay() { return monthlySalary; }  
}
```

```
Employee e = new FullTimeEmployee("An", 15_000_000);  
e.pay(); // runs FullTime version
```

1.7 Rules for Method Overriding

01 Same name and parameter list

Otherwise it is overloading, not overriding

02 Return type compatible

Same type or a subtype (covariant return)

03 Access cannot be stricter

protected → public is OK; public → private is not

04 Cannot override static / private / final

static hides; private is invisible; final is locked

05 Exceptions: no new checked ones

May throw fewer or more specific checked exceptions

06 Use @Override

Catch signature mistakes at compile time

1.7 Override vs Not Override

✓ Valid override

```
// Employee
public abstract double pay();
public String toString() { ... }

// FullTimeEmployee
@Override
public double pay() { return monthlySalary; }

@Override
public String toString() {
    return super.toString() + " (full-time)";
}
// same signature · specialize body
```

X Not an override

```
// Different params → overload
double pay(int months) { ... }
// not the same as pay()
// private → not visible
// cannot override a private method

// static → method hiding
static double taxRate() { ... }
// not polymorphic dispatch

// narrower access → illegal
// public pay() in Employee
// private pay() in subclass X
```

1.8 Final Methods and Classes

final method

Cannot be overridden in any subclass — locks the behavior

final class

Cannot be extended — no subclasses allowed (e.g. String)

final field / local

Different meaning: the variable cannot be reassigned

When to use

Security, immutability, design freeze, or avoid fragile overrides

1.8 final — Demo

```
public final class MathUtil { // no subclass
    public static int max(int a, int b) { ... }
}

public class Animal {
    public final void identity() { // cannot override
        System.out.println("I am an animal");
    }
}

// class Dog extends Animal {
//     void identity() { } ← compile error
// }
```

What If We Don't Want Behavior to Be Overridden?

Some methods must stay fixed — security checks, identity, invariants. Mark them final.

final method

Subclasses cannot override it — the algorithm is locked

When

Core rules that must not change in subtypes

Trade-off

Safety and clarity vs flexibility for future subtypes

What If We Don't Want Inheritance at All?

Sometimes the type itself must not be extended — String, utility holders, security-sensitive types.

final class

No subclasses allowed at all

Why

Immutability · security · avoid fragile base class problems

Alternative to inheritance

Prefer interfaces + composition when is-a is weak

Key Takeaways — Inheritance

- 01 Inheritance models is-a; subclass extends superclass
- 02 `super(...)` initializes the superclass part (first statement)
- 03 `protected` opens members to subclasses; `private` does not
- 04 Override = same signature, new behavior; use `@Override`
- 05 `final` method/class locks behavior or the type hierarchy
- 06 Next: Polymorphism — one type, many behaviors at runtime

02 · POLYMORPHISM

Polymorphism

Upcasting · Dynamic dispatch · instanceof · Collections

2.1 What Is Polymorphism?

Polymorphism ("many forms") means the same message can produce different behavior depending on the actual object type — decided at runtime.

One interface, many implementations

Call `pay()` on `Employee` — `FullTimeEmployee` and `PartTimeEmployee` calculate pay differently

Depends on inheritance / interfaces

Requires a shared supertype with overridable methods

Runtime decision

JVM looks at the real object, not the variable type

Enables flexible design

Write code against `Employee`; add new employee types without changing the calling code.

2.1 Same Call, Different Behavior

Employee *e*

e = new FullTimeEmployee(...)

e.pay()
→ monthlySalary

e = new PartTimeEmployee(...)

e.pay()
→ hourlyRate × hoursWorked

Reference type: Employee · Object type: FullTime or PartTime

2.2 Upcasting

Upcasting = treating a subclass object as its superclass type. Always safe; no cast syntax required. You only see members declared on the superclass.

Implicit and safe

Employee e = new FullTimeEmployee(...); //— Compiler allows it - No cast syntax required

View is narrowed

Through a you can call Employee methods, not FullTimeEmployee-only methods

Object stays a FullTimeEmployee

Runtime type is still FullTimeEmployee — overrides still run

Why useful?

Write general code once; work with any subclass uniformly

2.2 Upcasting — Demo

```
FullTimeEmployee f = new FullTimeEmployee(...);  
Employee e = f;           // upcast — implicit
```

Members available through e

```
e.pay();           // OK — method declared on Employee
```

```
// e.monthlySalary;
```

```
// ↑ compile error — monthlySalary not declared on Employee
```

```
// Actual object is still a FullTimeEmployee in the heap
```

2.2 Reference Type vs Object Type

Compile time

Looks at the
reference type
(Employee)

→ Which methods
are callable?

Runtime

Looks at the
object type
(FullTimeEmployee)

→ Which method
body actually runs?

2.3 Dynamic Method Dispatch

Dynamic method dispatch = the JVM chooses which overridden method to run based on the object's runtime type, not the reference type.

Late binding

The target method is resolved when the call happens, not at compile time

Requires override

Only instance methods that override participate; static methods do not

Core of polymorphism

Same source line `a.pay()` can run different bodies

Virtual method table

JVM uses the object's class to look up the correct implementation

2.3 Dispatch — Demo

```
void processPayroll(Employee e) {  
    e.pay();    // dispatch based on actual object  
}
```

```
processPayroll(new FullTimeEmployee(...));  
// → FullTimeEmployee.pay()
```

```
processPayroll(new PartTimeEmployee(...));  
// → PartTimeEmployee.pay()
```

```
// One method, three different behaviors
```

2.4 Polymorphism with Collections

Store different subclasses in one collection of the superclass type. Loop once; each element runs its own override.

List<Employee>

Can hold FulltimeEmployee, ParttimeEmployee... without separate lists

Uniform loop

for (Employee e : list) e.pay(); — each uses its version

Open for extension

Add a new subclass; old loop code still works

Typical pattern

UI widgets, game entities, payment methods, handlers...

2.4 Collections — Demo

```
List<Employee> list = new ArrayList<>();  
list.add(new FulltimeEmployee (...));  
list.add(new FulltimeEmployee (...));  
list.add(new ParttimeEmployee (...));  
  
for (Employee e : list) {  
    a.pay(); // FulltimeEmployee / ParttimeEmployee version each time  
}
```

2.5 Downcasting and instanceof

Downcasting = treat a superclass reference as a subclass type. Not always safe — wrong cast throws `ClassCastException`. Guard with `instanceof`.

When needed?

You must call a subclass-only method (ex. `e.getHoursWorked ()`)

`instanceof`

Check runtime type before casting

Pattern (Java 16+)

```
if (e instanceof ParttimeEmployee p) { e.getHoursWorked (); }
```

Prefer design

Often better to add a method on the superclass than to cast

2.5 Downcast — Demo

```
Employee e = new ParttimeEmployee("E101", "John Smith", 25.0, 80);

// Unsafe without check
// ParttimeEmployee p = (ParttimeEmployee) e; p.getHoursWorked ();

// Safe with instanceof (classic)
if (e instanceof ParttimeEmployee) {
    ParttimeEmployee p = (ParttimeEmployee) e;
    p.getHoursWorked ();
}

// Pattern matching (Java 16+)
if (e instanceof ParttimeEmployee p) {
    p.getHoursWorked ();
}
```

Key Takeaways — Polymorphism

- 01 Polymorphism: same call, behavior depends on runtime type
- 02 Upcasting is safe and implicit; narrows the visible API
- 03 Dynamic dispatch picks the override from the real object
- 04 Collections of a supertype hold mixed subclasses cleanly
- 05 Downcast only when needed; guard with `instanceof`
- 06 Next: Abstract Classes — incomplete types meant for extension

Demo Domain: Employee

Payroll needs one way to pay everyone, but Full-time and Part-time calculate pay differently.

Employee
(superclass)



FullTimeEmployee

monthly salary
+ benefits



PartTimeEmployee

hourly rate
× hours worked

Employee Hierarchy — Code

```
public abstract class Employee {
    protected String employeeId;
    protected String name;
    public Employee(String employeeId, String name) { this.employeeId = employeeId; this.name = name; }
    public abstract double pay(); // each subtype decides
}

public class FullTimeEmployee extends Employee {
    private double monthlySalary;
    public FullTimeEmployee(String id, String n, double s) { super(id, n); monthlySalary = s; }
    @Override public double pay() { return monthlySalary; }
}

public class PartTimeEmployee extends Employee {
    private double hourlyRate; private int hoursWorked;
    public PartTimeEmployee(String id, String n, double r, int h) {super(id, n); hourlyRate = r; hoursWorked = h; }
    @Override public double pay() { return hoursWorked * hourlyRate ; }
}
```

Upcasting & Dispatch — Employee

```
Employee e1 = new FullTimeEmployee("E101", "An", 15_000_000);  
Employee e2 = new PartTimeEmployee("E102", "Binh", 50_000, 80);
```

```
System.out.println(e1.pay()); // 15000000 (FullTime)  
System.out.println(e2.pay()); // 4000000 (PartTime)
```

```
// Same method name, different formulas
```

```
// Reference type: Employee
```

```
// Runtime type: FullTimeEmployee / PartTimeEmployee
```

Payroll with a Collection

```
List<Employee> staff = List.of(  
    new FullTimeEmployee("E101", "An", 15_000_000),  
    new PartTimeEmployee("E102", "Binh", 50_000, 80),  
    new FullTimeEmployee("E103", "Chi", 18_000_000)  
);
```

```
double total = 0;  
for (Employee e : staff) {  
    total += e.pay(); // polymorphic call  
    System.out.println(e.name + ": " + e.pay());  
}  
System.out.println("Total payroll: " + total);
```

When You Need Downcast

Prefer `pay()` on `Employee`. Downcast only if you need subtype-specific behavior that does not belong on the common API.

```
for (Employee e : staff) {  
    System.out.println(e.pay());    // preferred  
  
    if (e instanceof PartTimeEmployee p) {  
        System.out.println("Hours: " + p.getHoursWorked ());  
    }  
}
```

03 · ABSTRACT CLASSES

Abstract Classes

Incomplete types · Shared base · Forced specialization

3.1 Why Abstract Classes?

Sometimes a type should never be created by itself — only its specialized subtypes make sense. Abstract classes capture that idea.

No generic Employee object

In real payroll you always hire Full-time or Part-time — not a bare Employee

Shared structure

name, id, common methods still live in one place

Force subclasses to fill gaps

pay() must be implemented by every concrete employee type

Safe polymorphism

You can still write `List<Employee>` and call `pay()` on each

3.2 Defining Abstract Classes

abstract class

Declared with the abstract keyword — cannot be instantiated

May have fields & constructors

Subclasses call `super(...)` as usual

May mix abstract + concrete

Shared code stays concrete; gaps stay abstract

Must be extended

Useful work happens in concrete subclasses

3.2 abstract class Employee

```
public abstract class Employee {  
    protected String employeeId;  
    protected String name;  
  
    public Employee(String employeeId, String name) {  
        this.employeeId = employeeId;  
        this.name = name;  
    }  
  
    public String getName() { return name; } // concrete  
  
    public abstract double pay(); // no body — subclass must implement  
}  
  
// Employee e = new Employee("E101", "An"); ← compile error
```

3.3 Abstract Methods

An abstract method has a signature but no body. Any concrete subclass must override it (or stay abstract itself).

Declared with abstract

`public abstract double pay();` — ends with semicolon

No method body

Implementation is deferred to subclasses

Only in abstract classes

A concrete class cannot declare abstract methods

Contract for subtypes

FullTime and PartTime must each provide `pay()`

3.3 Implementing pay()

FullTimeEmployee

```
public class FullTimeEmployee extends Employee {
    private double monthlySalary;

    public FullTimeEmployee( String id,
        String n, double s) {
        super(id, n);
        this.monthlySalary = s;
    }

    @Override
    public double pay() {
        return monthlySalary;
    }
}
```

PartTimeEmployee

```
public class PartTimeEmployee
    extends Employee {
    private double hourlyRate;
    private int hoursWorked;

    public PartTimeEmployee( String id,
        String n, double r, int h) {
        super(id, n); hourlyRate = r; hoursWorked = h;
    }

    @Override
    public double pay() {
        return hourlyRate * hoursWorked;
    }
}
```

3.4 Concrete Methods

An abstract class can still hold fully implemented methods. Shared behavior lives once in the base; subclasses inherit it for free.

Not every method is abstract

Only the parts that differ by subtype need to be abstract

Reuse without copy-paste

getName(), validate() can be concrete on Employee

May call abstract methods

A concrete method can use pay() — resolved at runtime in the subclass

Template-style design

Common steps in the base; specialized steps in overrides

3.4 Concrete + Abstract Together

```
public abstract class Employee {
    protected String employeeId;
    protected String name;
    public Employee(String employeeId, String name) {
        this.employeeId = employeeId;
        this.name = name;
    }
    public String getName() { return name; }    // concrete
    @Override
    public String toString() {                  // concrete
        System.out.println("employeeId - " + name + ": " + pay()); // calls abstract
    }
    public abstract double pay();              // abstract
}
// FullTime / PartTime only implement pay()
// toString() works for both automatically
```

3.5 Constructors in Abstract Classes

Abstract classes can have constructors

They initialize fields shared by all subclasses

You never call them with new

Only subclasses invoke them via `super(...)`

Same rules as normal constructors

`super(...)` must be the first statement in the subclass constructor

Guarantees valid base state

Every `FullTime` / `PartTime` starts with a proper name (and other shared data)

3.5 Constructor Chain — Demo

```
public abstract class Employee {  
    protected String employeeId;  
    protected String name;  
    public Employee(String employeeId, String name) {  
        this.employeeId = employeeId;  
        if (name == null || name.isBlank()) throw new IllegalArgumentException("name required");  
        this.name = name;  
    }  
}  
  
public class FullTimeEmployee extends Employee {  
    public FullTimeEmployee(String employeeId, String name, double salary) {  
        super(employeeId, name); // runs Employee constructor first  
        this.monthlySalary = salary;  
    }  
}
```

3.5 Construction Order

1

new FullTimeEmployee(...)

Memory allocated for the full object

2

Employee constructor runsname validated and assigned via `super(name)`

3

FullTimeEmployee body runs

monthlySalary (and other subclass fields) assigned

4

Reference returnedObject is ready — usable as `Employee` or `FullTime`

3.6 Abstract Classes and Polymorphism

You cannot new an abstract class, but you can use it as a reference type. Polymorphism works the same: call `pay()` through `Employee`, runtime runs `FullTime` or `PartTime`.

Reference type can be abstract

`Employee e = new FullTimeEmployee(...);` — legal

Collections of abstract type

`List<Employee>` holds any concrete subtype

Dispatch still dynamic

`e.pay()` selects the override from the real object

Design benefit

API depends on `Employee`; new subtypes plug in without changing callers

3.6 Polymorphism with abstract Employee

```
List<Employee> staff = List.of(  
    new FullTimeEmployee("E101", "An", 15_000_000),  
    new PartTimeEmployee("E102", "Binh", 50_000, 80)  
);  
  
for (Employee e : staff) {  
    System.out.println(e); // concrete on abstract class  
    // inside: uses e.pay() → FullTime or PartTime  
}  
  
// No new Employee(...) anywhere — only subtypes
```

Key Takeaways — Abstract Classes

- 01 Abstract class = incomplete type; cannot be instantiated
- 02 Abstract methods force every concrete subclass to implement them
- 03 Concrete methods share common behavior (`getName`, `pay()`)
- 04 Constructors initialize shared state via `super(...)`
- 05 Still fully polymorphic through abstract reference types
- 06 Next: Interfaces — pure contracts, multiple inheritance of type

04 · INTERFACES

Interfaces

Contracts · Multiple types · Default & static methods

4.1 What Is an Interface?

An interface is a contract: it names what a type can do, not how. Classes implement the contract; callers depend on the interface, not the concrete class.

Pure capability

Payable means “can compute a payment” — no fields required

Multiple inheritance of type

A class can implement many interfaces at once

No instance state (classic)

Before Java 8: only abstract methods + constants

Decoupling

Payroll code talks to Payable; FullTime / contractor / vendor can all fit

4.1 Example: Payable

```
public interface Payable {  
    double pay(); // every implementor must provide this  
}
```

```
// Later: Employee, Invoice, Contractor...  
// all can implement Payable and join payroll
```

```
void process(Payable p) {  
    System.out.println("Paying: " + p.pay());  
}
```

4.2 Interface Types

Reference type

`Payable p = new FullTimeEmployee(...);` — upcast to interface

Parameter / return type

Methods can accept or return interface types for flexibility

Collection element type

`List<Payable>` mixes employees, invoices, any implementor

Not instantiable

Cannot `new Payable()` — only concrete implementors

4.2 Interface as a Type — Demo

```
Payable p1 = new FullTimeEmployee("E101", "An", 15_000_000);  
Payable p2 = new PartTimeEmployee("E102", "Binh", 50_000, 80);  
  
List<Payable> payments = List.of(p1, p2);  
for (Payable p : payments) {  
    System.out.println(p.pay()); // dispatch on real class  
}  
  
// process() only needs Payable — not Employee
```

4.3 Defining & Implementing Interfaces

Define with interface

```
public interface Payable { double pay(); }
```

Implement with implements

```
class FullTimeEmployee extends Employee implements Payable
```

Provide every abstract method

Or the class must be declared abstract itself

public methods

Interface methods are public abstract by default (pre-default methods)

4.3 Payable + Employee — Code

```
public interface Payable {  
    double pay();  
}
```

```
public abstract class Employee implements Payable {  
    protected String employeeId;  
    protected String name;  
    public Employee(String employeeId, String name) { this.employeeId = employeeId;  
        this.name = name; }  
    public abstract double pay(); // pay() still abstract — concrete subclasses implement it  
}
```

```
public class FullTimeEmployee extends Employee {  
    private double monthlySalary;  
    @Override public double pay() { return monthlySalary; }  
}
```

4.4 Interface References

A variable of interface type can refer to any object whose class implements that interface. Only interface members are visible through that reference.

Upcast to interface

`Payable p = new FullTimeEmployee(...);` — implicit and safe

Visible API is the contract

`p.pay()` OK; `p.getName()` only if also declared on `Payable`

Runtime type unchanged

Object is still `FullTimeEmployee`; overrides still dispatch

Programming to the interface

Callers depend on `Payable`, not on a specific class

4.4 Interface Reference — Demo

```
FullTimeEmployee ft = new FullTimeEmployee("E101", "An", 15_000_000);
```

```
Payable p = ft; // interface reference
```

```
System.out.println(p.pay()); // OK — on Payable
```

```
// System.out.println(p.getName());
```

```
// ↑ only if getName() is part of Payable
```

```
void settle(Payable p) {
```

```
    System.out.println("Transfer: " + p.pay());
```

```
}
```

```
settle(ft); // any Payable works
```

4.5 Multiple Interfaces

A class may implement several interfaces. That is how Java models multiple inheritance of type — one class, many roles.

implements A, B, C

List every interface; implement all of their methods

Separate concerns

Payable for money · Identifiable for id · Taxable for tax code

One object, many views

Same employee as Payable in payroll and Identifiable in HR

No diamond of state

Interfaces (mostly) have no instance fields — simpler than multiple class inheritance

4.5 Multiple Interfaces — Demo

```
public interface Payable { double pay(); }
public interface Identifiable { String getId(); }

public class FullTimeEmployee extends Employee
    implements Payable, Identifiable {
    private String id;
    @Override public double pay() { return monthlySalary; }
    @Override public String getId() { return id; }
}

Payable pay = emp;    // role: payment
Identifiable who = emp; // role: identity
```

4.5 One Object, Several Roles

as Payable

pay()

as Identifiable

getId()

as Employee

getName()

FullTimeEmployee
instance

Same object, different reference types → different visible methods.
Choose the interface that matches the collaboration (payroll vs HR vs reporting).

4.6 Extending Interfaces

An interface can extend one or more other interfaces. The child interface inherits all abstract methods (and defaults) from its parents.

extends (not implements)

```
interface Taxable extends Payable { double tax(); }
```

Multiple parents allowed

```
interface X extends A, B, C { }
```

Implementor must satisfy all

Class implementing Taxable must provide pay() and tax()

Refine contracts

Start general (Payable), specialize (Taxable, BonusPayable)

4.6 Extending Interfaces — Demo

```
public interface Payable {  
    double pay();  
}  
  
public interface Taxable extends Payable {  
    double tax(); // plus inherited pay()  
}  
  
public class FullTimeEmployee extends Employee  
    implements Taxable {  
    @Override public double pay() { return monthlySalary; }  
    @Override public double tax() { return monthlySalary * 0.1; }  
}
```

4.7 Default, Static & Private Methods

default methods (Java 8)

Provide a body in the interface; implementors may override

static methods (Java 8)

Belong to the interface type; call `InterfaceName.method()`

private methods (Java 9)

Helpers inside the interface for default/static code only

Why?

Evolve APIs without breaking all implementors; share small utilities

4.7 default / static — Demo

```
public interface Payable {  
    double pay();  
  
    default String payslip() { // optional override  
        return "Amount: " + format(pay());  
    }  
  
    static String format(double amount) { // interface-level  
        return String.format("%.0f VND", amount);  
    }  
}
```

```
Payable p = new FullTimeEmployee("An", 15_000_000);  
System.out.println(p.payslip());    // default method  
System.out.println(Payable.format(100)); // static
```

Key Takeaways — Interfaces

- 01 Interface = contract of capabilities, not a concrete implementation
- 02 Classes implement interfaces; references can use the interface type
- 03 A class may implement many interfaces (multiple roles)
- 04 Interfaces can extend other interfaces to refine contracts
- 05 default / static / private methods help evolve and organize APIs
- 06 Next: Comparison & Design — when abstract class vs interface

05 · COMPARISON & DESIGN

Comparison & Design

Abstract vs Interface · Inheritance vs Interface · When to use what

5.1 Abstract Class vs Interface

Aspect	Abstract class	Interface
Instantiation	No (if abstract)	No
Fields / state	Instance fields OK	Constants; no instance state*
Constructors	Yes	No
Methods	Abstract + concrete	Abstract + default/static
Inheritance	extends one class	implements many
Access modifiers	Any	public (mostly)
Purpose	Shared base + is-a	Capability / contract

5.1 In the Employee Domain

Abstract class Employee

- Shared fields: name, id
- Constructor validation
- Concrete helpers: printPayslip()
- Abstract pay()
- is-a hierarchy

FullTime / PartTime

Interface Payable

- Only capability: pay()
- No state
- Employee implements it
- Invoice / Vendor can too
- Payroll depends on
Payable, not Employee

5.2 Inheritance vs Interface

Inheritance expresses is-a and shared implementation. Interfaces express can-do / roles. Prefer the weaker coupling when is-a is not true.

is-a relationship?

Yes → consider inheritance (Dog is an Animal)

Only a capability?

Yes → interface (can be paid, can be sorted, can be compared)

Need shared fields/ctors?

Abstract class fits better than interface alone

Need multiple parents?

Interfaces — a class cannot extend two classes

5.2 Choose the Right Relationship

X Weak design

Invoice extends Employee
just to reuse pay()

- False is-a
- Rigid hierarchy
- Hard to change later

✓ Better design

Employee implements Payable
Invoice implements Payable

- True roles
- Payroll uses Payable
- Easy to extend

5.3 When to Use Inheritance

Clear is-a relationship

FullTimeEmployee is an Employee; Dog is an Animal

Shared state and constructors

name, id, validation — one place in the superclass

Shared algorithms with hooks

Template steps in the base; subclasses fill abstract methods

Stable hierarchy

The set of subtypes is relatively fixed and domain-driven

You need protected access

Subclasses legitimately share internal details of the base

5.3 Inheritance — Use with Care

Avoid for pure code reuse

If there is no is-a, prefer composition or an interface

Deep trees are fragile

Changes high in the hierarchy break many subclasses

One superclass only

Cannot extend two classes — use interfaces for extra roles

Exposes internals

protected couples subclasses to base implementation details

5.4 When to Use Interfaces

Capability / role, not identity

Payable, Comparable, Serializable, Runnable

Multiple unrelated implementors

Employee, Invoice, Vendor all can be Payable

API boundary

Depend on Payable in payroll — hide concrete classes

Multiple roles on one type

implements Payable, Identifiable, Taxable

Plugin / extension points

New payment kinds without changing existing callers

5.4 In Practice (Employee Domain)

Use inheritance

Employee ← FullTime

Employee ← PartTime

Shared name, constructor,
pay

Use interface

Payable for pay()

Taxable for tax()

Also: Invoice, Contractor

Combine both

abstract class Employee
implements Payable

Best of shared base + role

5.5 Putting Everything Together

One coherent design: inheritance for is-a, abstract base for shared structure, interfaces for roles, polymorphism for flexible use.

Payable

```
interface  
    pay()
```

Employee

```
abstract class  
    employeeId  
    name  
    abstract pay()
```

FullTime

```
extends Employee  
pay() = monthlySalary
```

PartTime

```
extends Employee  
    pay() =  
        hourlyRate×hoursWorked
```

Employee implements Payable · FullTime / PartTime extend Employee
Callers use List<Payable> or List<Employee> — new subtypes plug in without changing payroll code.

5.5 Complete Sketch

```
public interface Payable { double pay(); }
```

```
public abstract class Employee implements Payable {  
    protected String employeeId;  
    protected String name;  
    public Employee(String employeeId, String name) { this.employeeId = employeeId; this.name = name; }  
    @Override  
    public String toString() { System.out.println(employeeId + "-" + name + ": " + pay()); }  
    public abstract double pay();  
}
```

```
public class FullTimeEmployee extends Employee {  
    private double monthlySalary;  
    public FullTimeEmployee(String id, String n, double s) { super(id, n); monthlySalary = s; }  
    @Override public double pay() { return monthlySalary; }  
}
```

```
public class PartTimeEmployee extends Employee { /* hourlyRate * hoursWorked */ }
```

5.5 Using the Design

```
List<Payable> payments = List.of(  
    new FullTimeEmployee("E101", "An", 15_000_000),  
    new PartTimeEmployee("E102", "Binh", 50_000, 80)  
    // + Invoice, Contractor... later  
);  
  
double total = 0;  
for (Payable p : payments) {  
    total += p.pay();      // polymorphism  
}  
  
for (Employee e : List.of(/* employees */)) {  
    System.out.println(e); // abstract + concrete  
}
```

Key Takeaways — Comparison & Design

- 01 Abstract class = shared base + is-a; Interface = capability / role
- 02 Inheritance when true is-a and shared state; interfaces for can-do
- 03 Prefer depending on interfaces at API boundaries
- 04 Combine: abstract Employee implements Payable
- 05 Avoid false is-a just to reuse code — use interfaces or composition
- 06 Design for extension: new subtypes without rewriting callers

Chapter Summary

01**Inheritance**

extends · super · override · protected · final

02**Polymorphism**

Upcast · dispatch · collections · instanceof

03**Abstract Classes**

Incomplete types · abstract methods · shared base

04**Interfaces**

Contracts · multiple roles · default methods

05**Design**

is-a vs can-do · abstract + interface together

End of Chapter

Inheritance · Polymorphism

Abstract Classes · Interfaces

Review the Employee example · Practice small hierarchies · Prefer clear is-a and clean contracts